



Pluton Ton Bridge Security Audit Report

December 3, 2025



Contents

1 Introduction

[1.1 About Pluton Bridge](#)

[1.2 Source Code](#)

[1.3 Revision History](#)

2 Overall Assessment

3 Vulnerability Summary

[3.1 Overview](#)

[3.2 Security Level Reference](#)

[3.3 Vulnerability Details](#)

4 Appendix

[4.1 About AstraSec](#)

[4.2 Disclaimer](#)

[4.3 Contact](#)

1 Introduction

1.1 About Pluton Bridge

Pluton Bridge on TON is a cross-chain bridge connecting TON with other blockchains, supporting native TON and Jetton tokens. It uses an intent-based event model: users send assets to Pluton and the contract emits events for off-chain verifiers. Claiming on the destination chain uses signed claims and the Claim Storage contract. The design separates intent recording from execution via signed verification and state tracking to improve security.



1.2 Source Code

The following source code was reviewed during the audit:

- ▶ <https://github.com/pluton-bridge/pluton-ton-contract/tree/v3-func>
- ▶ CommitID: 50cda6f6dc3371545abb8c327432df9a883418fc

And this is the final version representing all fixes implemented for the issues identified in the audit:

- ▶ <https://github.com/pluton-bridge/pluton-ton-contract/tree/v3-func>
- ▶ CommitID: 8520c098601e3ce430605a36361f15bf8649e3d8

1.3 Revision History

Version	Date	Description
v1.0	December 3, 2025	Initial Audit

2 Overall Assessment

This report has been compiled to identify issues and vulnerabilities within the Pluton protocol. Throughout this audit, we identified a total of 5 issues spanning various severity levels. By employing auxiliary tool techniques to supplement our thorough manual code review, we have discovered the following findings.

Severity	Count	Acknowledged	Won't Do	Addressed
Critical	—	—	—	—
High	—	—	—	—
Medium	4	1	—	3
Low	1	—	—	1
Informational	—	—	—	—
Total	5	1	—	4

3 Vulnerability Summary

3.1 Overview

Click on an issue to jump to it, or scroll down to see them all.

M-1

[Improper msg_value Refund Causing Balance Depletion](#)

M-2

[Potential Signature Replay in recv_claim_request\(\)](#)

M-3

[Double Decoding of query_id in op::burn_notification\(\) Handler](#)

M-4

[Potential Risks Associated with Centralization](#)

L-1

[Improved Handling of Bounced op::internal_transfer](#)

3.2 Security Level Reference

In web3 smart contract audits, vulnerabilities are typically classified into different severity levels based on the potential impact they can have on the security and functionality of the contract. Here are the definitions for critical-severity, high-severity, medium-severity, and low-severity vulnerabilities:

Severity	Acknowledged
C-X (Critical)	A severe security flaw with immediate and significant negative consequences. It poses high risks, such as unauthorized access, financial losses, or complete disruption of functionality. Requires immediate attention and remediation.
H-X (High)	Significant security issues that can lead to substantial risks. Although not as severe as critical vulnerabilities, they can still result in unauthorized access, manipulation of contract state, or financial losses. Prompt remediation is necessary.
M-X (Medium)	Moderately impactful security weaknesses that require attention and remediation. They may lead to limited unauthorized access, minor financial losses, or potential disruptions to functionality.
L-X (Low)	Minor security issues with limited impact. While they may not pose significant risks, it is still recommended to address them to maintain a robust and secure smart contract.
I-X (Informational)	Warnings and things to keep in mind when operating the protocol. No immediate action required.
U-X (Undetermined)	Identified security flaw requiring further investigation. Severity and impact need to be determined. Additional assessment and analysis are necessary.

3.3 Vulnerability Details

3.3.1 [M-1] Improper msg_value Refund Causing Balance Depletion

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
pluton.fc	Business Logic	Medium	Medium	Addressed

In the handling of `prefix::pluton_unlock_toncoin` message in `pluton` contract, the entire `msg_value` is refunded directly to the user via `send_pluton_excesses()`. This does not deduct required fees (e.g., compute fees, forward fees), which are instead charged from the contract's balance. Additionally, the `send_exact_native_ton_to_address()` function uses `SEND_MODE_PAY_FEES_SEPARATELY` flag, further deducting forward fees from the contract balance.

This pattern also occurs in the handling of `prefix::pluton_claim_from_storage` and `prefix::pluton_deploy_claim_storage` messages, where `msg_value` is refunded in full without reserving fees, risking the same balance depletion. As a result, the contract's balance may fall below `min_tons_for_pluton`, triggering a frozen state and halting all operations.

pluton-ton-contract - pluton.fc

```
234 if (op == prefix::pluton_unlock_toncoin){
235     require_admin(msg_sender_address);
236     int msg_native_amount = in_msg_body~load_coins();
237     slice native_receiver = in_msg_body~load_msg_addr();
238     ;; if amount drains pluton, prevent it
239     throw_if(exit_code::withdraw_exceeding, msg_native_amount > my_balance - min_tons_for_pluton);
240     ;; send native to given address
241     send_exact_native_ton_to_address(native_receiver, msg_native_amount);
242     send_pluton_excesses(msg_sender_address, msg_value);
243     return ();
244 }
```

Remediation Properly deduct all fees from `msg_value` before refunding excess. Apply this fix consistently across `pluton_unlock_toncoin`, `pluton_claim_from_storage`, and `pluton_deploy_claim_storage` handlers. Reference `receive_tokens()` in `jetton_wallet` for correct excess refund logic.

3.3.2 [M-2] Potential Signature Replay in `recv_claim_request()`

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
claim_storage.fc	Signature Replay	Medium	Medium	Addressed

The `recv_claim_request()` function in the `claim_storage` contract is used to process `prefix::pluton_storage_claim` messages, which allow users to claim Jettons from pluton using an admin-signed request. The signature is verified using `check_signature` on a hash constructed from claim parameters (amount, Jetton wallet, claimer address, claim type, gas values, etc.). However, the hash lacks a domain separator — specifically, it does not include the workchain ID and the contract address (`my_address()`). As a result, a valid signature generated for this contract can be replayed on:

- The same contract in a different workchain.
- A different contract with similar logic in the same or another workchain.

This violates the principle of domain separation in cryptographic signatures (similar to EIP-712), enabling cross-contract and cross-chain replay attacks.

```
pluton-ton-contract - claim_storage.fc

42 slice check_data_slice = begin_cell()
43   .store_coins(amount) ;; claim amount
44   .store_slice(pluton_jetton_wallet_address) ;; jetton wallet address
45   .store_ref( ;; claim info
46     begin_cell()
47       .store_slice(claimer_address)
48       .store_uint(claim_type, size::claim_type)
49       .store_coins(claim_gas) ;; claim gas provided by admin signature
50     .end_cell()
51   )
52   .store_coins(claim_gas) ;; gas for claim of jetton, which will be sent and used in pluton and is provided by admin
53   .store_coins(total_claimed_in_msg)
54   .end_cell().begin_parse();
55
56
57 ;; check if signature matches with data
58 int is_signature_valid = check_signature(slice_hash(check_data_slice), admin_signature, owner_public_key);
59 throw_unless(exit_code::invalid_signature, is_signature_valid == TRUE);
60
```

Remediation Include the workchain ID and contract address in the signed data hash to bind the signature to this specific contract instance and network.

3.3.3 [M-3] Double Decoding of query_id in op::burn_notification() Handler

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
jetton_minter.fc	Coding Practice	Medium	Medium	Addressed

The jetton_minter contract incorrectly decodes the query_id field twice when processing an incoming op::burn_notification() message. The first decoding occurs immediately after the op code:

```
pluton-ton-contract - jetton_minter.fc
59 int op = in_msg_body~load_uint(32);
60 int query_id = in_msg_body~load_uint(64);
```

However, in the op::burn_notification branch, query_id is loaded again (line 95). This misaligns the message body slice, causing jetton_amount and from_address to be parsed from the wrong bits. Consequently, the sender wallet address validation fails for all valid requests. As a result, all legitimate burn requests are rejected, rendering the Jetton burn functionality completely non-operational.

```
pluton-ton-contract - jetton_minter.fc
94 if (op == op::burn_notification()) {
95     int query_id = in_msg_body~load_uint(64);
96     int jetton_amount = in_msg_body~load_coins();
97     slice from_address = in_msg_body~load_msg_addr();
98     throw_unless(74,
99         equal_slices_bits(calculate_user_jetton_wallet_address(from_address, my_address(), jetton_wallet_code), sender_address)
100     );
101     ...
102 }
```

Remediation Remove the duplicate query_id decoding in the burn_notification branch. The query_id should only be loaded once after the op code.

3.3.4 [M-4] Potential Risks Associated with Centralization

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
Multiple Contracts	Security	Medium	Low	Acknowledged

The pluton contract grants extensive protocol-wide control to a single privileged owner_address, introducing significant centralization risks. Specifically, this account holds authority to:

- Unlock native TON or Jettons from the contract via pluton_unlock_toncoin and pluton_unlock_jetton operations.
- Sign user claim requests, enabling asset withdrawals through the pluton_storage_claim mechanism.

This concentration of control in a limited number of entities undermines the protocol's decentralized design and exposes it to potential vulnerabilities.

```
pluton-ton-contract - pluton.fc
251 if (op == prefix::pluton_unlock_jetton){
252     int query_id = in_msg_body~load_uint(size::query_id);
253     int jetton_amount = in_msg_body~load_coins();
254     slice jetton_receiver = in_msg_body~load_msg_addr(); ;; dest_addr
255     slice jetton_wallet_address = in_msg_body~load_msg_addr(); ;; pluton wallet_addr
256
257     require_admin(msg_sender_address);
258
259     send_jetton_without_forward_payload(
260         query_id, jetton_wallet_address, jetton_receiver, jetton_amount, msg_sender_address, null(), SEND_MODE_CARRY_ALL_REMAINING_MESSAGE_VALUE, 0
261     );
262     return ();
263 }
```

Remediation To mitigate the identified issue, it is recommended to introduce multi-sig mechanism to undertake the role of the privileged accounts. Moreover, it is advisable to implement timelocks to govern all modifications to the privileged operations.

Response By Team This issue has been acknowledged by the team.

3.3.5 [L-1] Improved Handling of Bounced op::internal_transfer

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
jetton_minter.fc	Business Logic	Low	Low	Addressed

jetton_minter contract enables the owner to mint Jettons via op::mint(), which increases total_supply and sends a bounceable op::internal_transfer() message to credit the destination wallet. However, if the op::internal_transfer() bounces, the contract silently discards the bounced message by returning early when flags & 1 is set. As a result, total_supply remains permanently increased while the user receives no tokens, creating an irreversible accounting mismatch. This inflates the reported total supply without actual token issuance, violating the Jetton standard and undermining token integrity.

```
pluton-ton-contract - jetton_minter.fc
46  if (in_msg_body.slice_empty?()) { ;; ignore empty messages
47      return ();
48  }
49  slice cs = in_msg_full.begin_parse();
50  int flags = cs~load_uint(4);
51
52  if (flags & 1) { ;; ignore all bounced messages
53      return ();
54  }
55  slice sender_address = cs~load_msg_addr();
56  int fwd_fee_from_in_msg = cs~retrieve_fwd_fee(); ;; get fwd_fee from in_msg_full
57  int fwd_fee = get_original_fwd_fee(MY_WORKCHAIN, fwd_fee_from_in_msg); ;; we use message fwd_fee for estimation of forward_payload costs
58
```

Remediation Properly handle the bounced op::internal_transfer() messages originating from a mint operation, extract the intended mint amount from the bounced body, and decrement total_supply accordingly.

4 Appendix

4.1 About AstraSec

AstraSec is a blockchain security company that serves to provide high-quality auditing services for blockchain-based protocols. With a team of blockchain specialists, AstraSec maintains a strong commitment to excellence and client satisfaction. The audit team members have extensive audit experience for various famous DeFi projects. AstraSec's comprehensive approach and deep blockchain understanding make it a trusted partner for the clients.

4.2 Disclaimer

The information provided in this audit report is for reference only and does not constitute any legal, financial, or investment advice. Any views, suggestions, or conclusions in the audit report are based on the limited information and conditions obtained during the audit process and may be subject to unknown risks and uncertainties. While we make every effort to ensure the accuracy and completeness of the audit report, we are not responsible for any errors or omissions in the report.

We recommend users to carefully consider the information in the audit report based on their own independent judgment and professional advice before making any decisions. We are not responsible for the consequences of the use of the audit report, including but not limited to any losses or damages resulting from reliance on the audit report.

This audit report is for reference only and should not be considered a substitute for legal documents or contracts.

4.3 Contact

Phone	+86 156 0639 2692
Email	contact@astrasec.ai
Twitter	https://x.com/AstraSecAI